MINI PROJECT I · T.E. SEMESTER V · DEPARTMENT OF COMPUTER ENGINEERING · SPIT · GROUP 20

EcoMesh

A peer-to-peer communication mesh for disaster relief: an SOS hops from device to device until it reaches a rescue team

Twissha Shah · **Miti Shah** · **Tanvi Sheth** | Guide: Prof. Anuj Tawari

The problem

After an earthquake, fire or building collapse, cell towers fail or are jammed. Messaging apps need a server far away, so they stop even for two people in the same building.

But phones still have battery, and nearby devices can still reach each other. People trapped inside cannot tell anyone where they are, and rescuers search room by room.

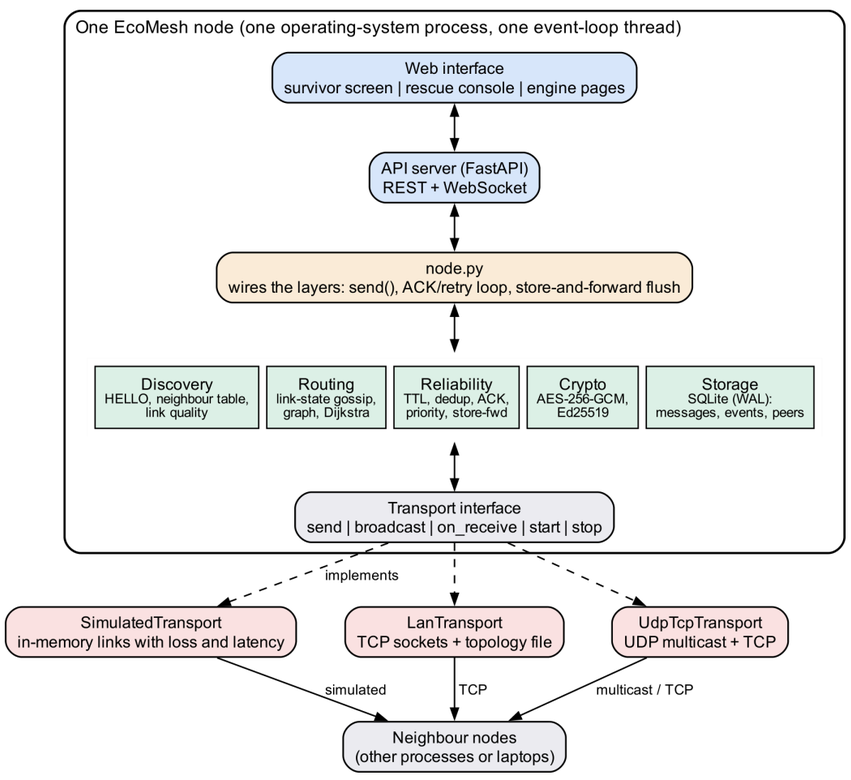
The idea

Make every device both an endpoint and a relay. An SOS travels from phone to phone until it reaches a rescue team or a gateway to the outside. No tower, no internet, no central server.

Messages are encrypted (AES-256-GCM), identities are signed (Ed25519), and a message with no route is held and sent the moment a path appears.

The survivor picks one of **nine request types** (Medical emergency, Injured, Trapped, Fire or smoke, Unsafe building, Need water, Need food, Need medicine, Elderly or child needs help) and can **speak** the location instead of typing it.

Architecture and how an SOS travels



One node = one process on one event-loop thread. Every layer reaches the network only through the transport interface, so the same engine runs on a simulated radio, on TCP between processes, or on UDP.

1. **Route.** Dijkstra over a link-state graph that every node builds from its neighbours' reports.
2. **Seal.** The payload is encrypted and the packet's identity is signed.
3. **Relay.** Every hop sends SOS before routine traffic.
4. **Confirm.** The receiver acknowledges; a missed ACK retries on a recomputed route.
5. **Hold.** No route? The message is kept and sent when a path appears.

$$\text{cost} = 1 + 5\,(1 - q) + 3\,f$$

q: share of recent beacons heard; f: recent ACK timeouts. A lossy link costs more than two clean hops, so routes avoid it.

Measured results

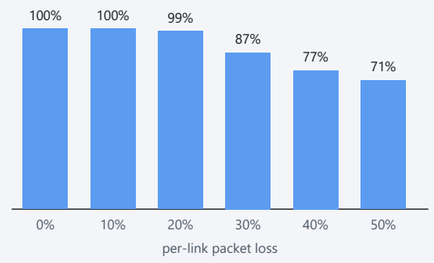
92.6%
delivered on a 40-node mesh at 20% packet loss, with relay and gateway failures

4.4×
faster median delivery for an SOS than routine traffic under load

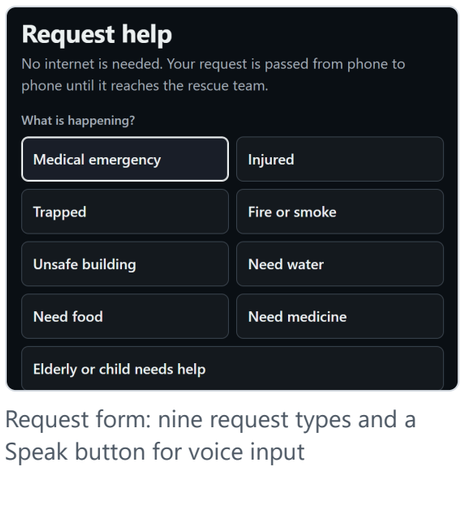
99% vs 57%
weighted routing against hop-count routing on identical seeds

334
automated tests pass; 8 of 10 target metrics fully met

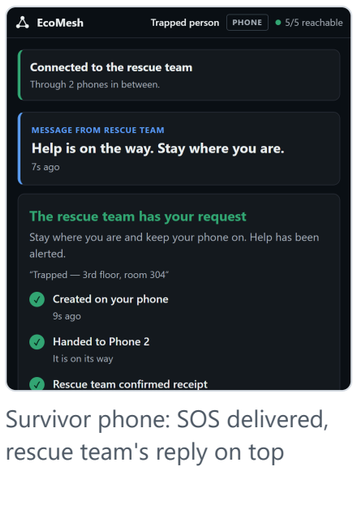
Delivery ratio as packet loss rises (21-node mesh, 10 runs each)



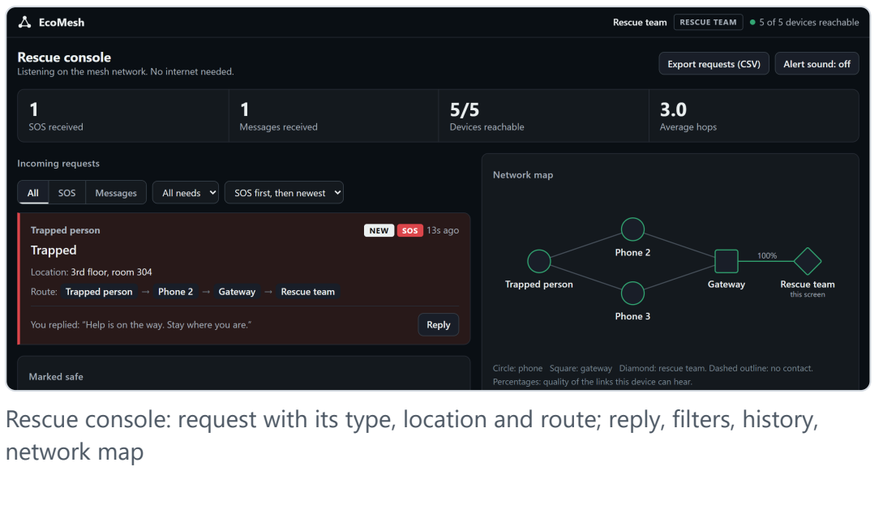
One app for the whole demonstration, and what it does not do



Request form: nine request types and a Speak button for voice input



Survivor phone: SOS delivered, rescue team's reply on top



Rescue console: request with its type, location and route; reply, filters, history, network map

Limits

- The radio is emulated: real packets over TCP, between configured neighbours only.
- No phone app: a phone opens a web page served by a laptop.
- Voice input uses the browser's online speech service; typing always works.
- One shared key protects the mesh.
- Not fully met: convergence; recovery after relay death.

Future work

- Bluetooth LE or Wi-Fi Direct transport and a phone client.
- Per-pair key agreement (X25519).
- Adaptive timers; a usability study.

Code, report and testing document

https://github.com/tanvisheth1234-wq/miniproject

40-page report · 10 target metrics validated against stored measurements · every number is generated from the code